

Inference Systems

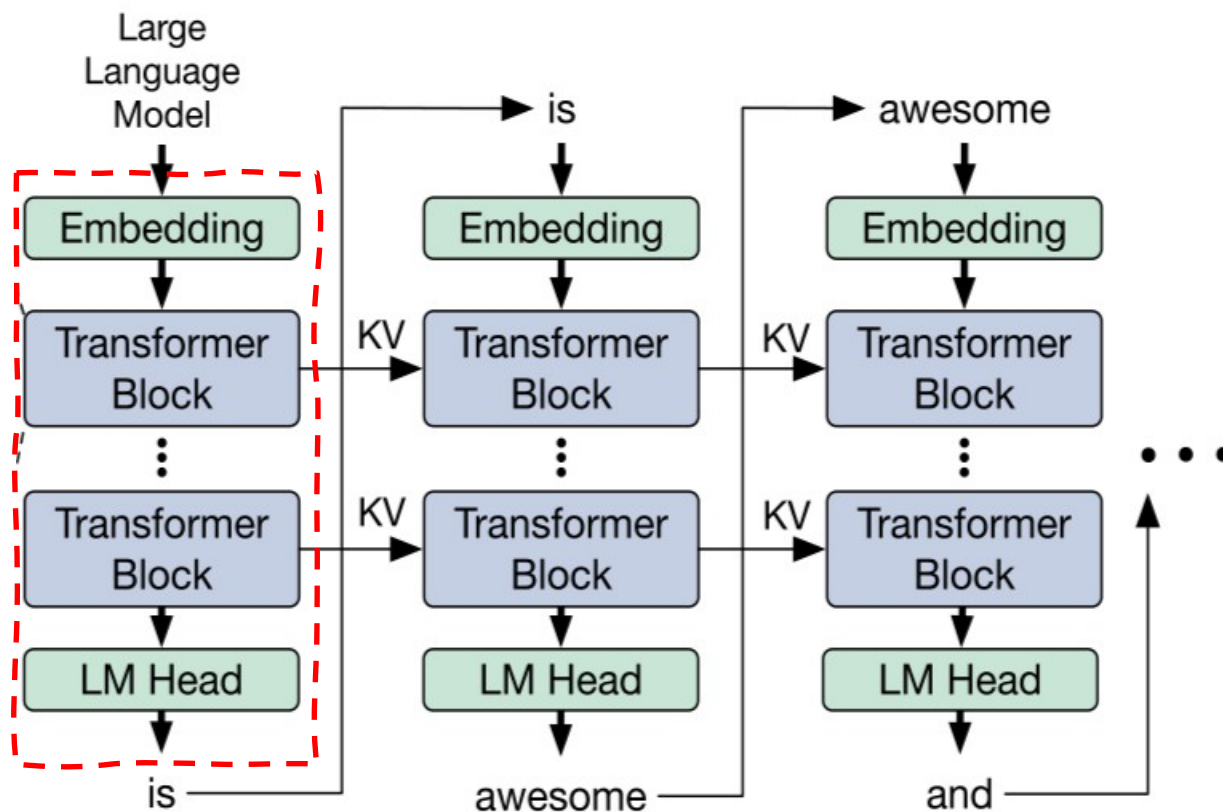
Outline

1. Background
2. Core Features – KV Caching, Batching, Paged Attention
3. Decoding
4. Small Inference Demo

What is inference?

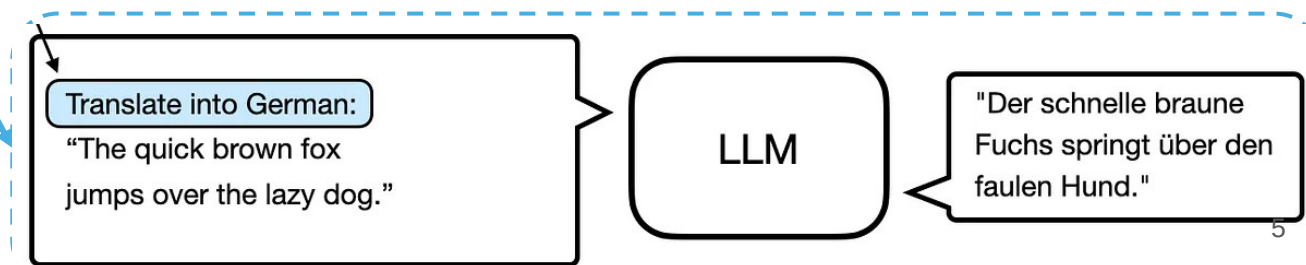
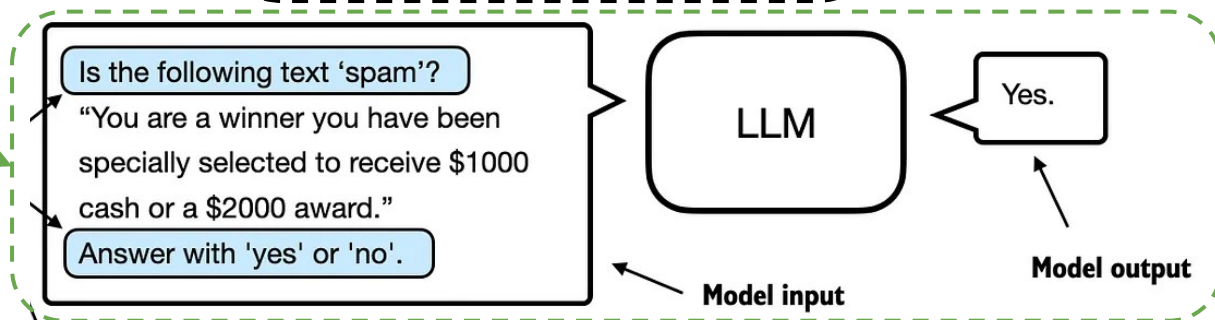
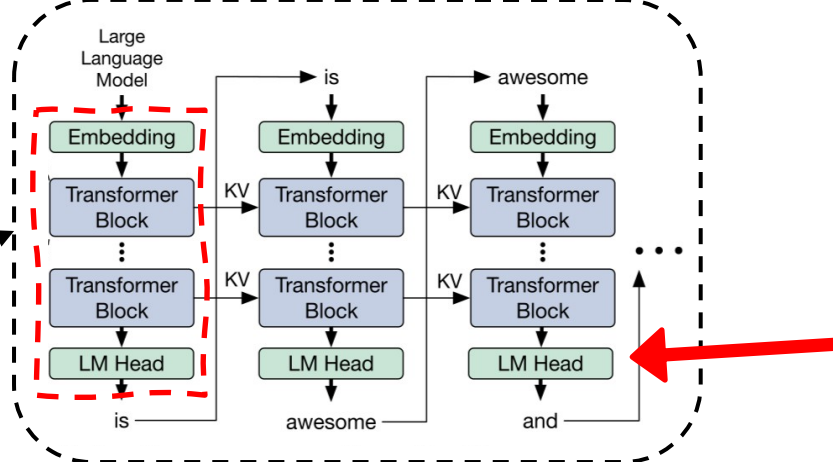
What is inference?

Text generation

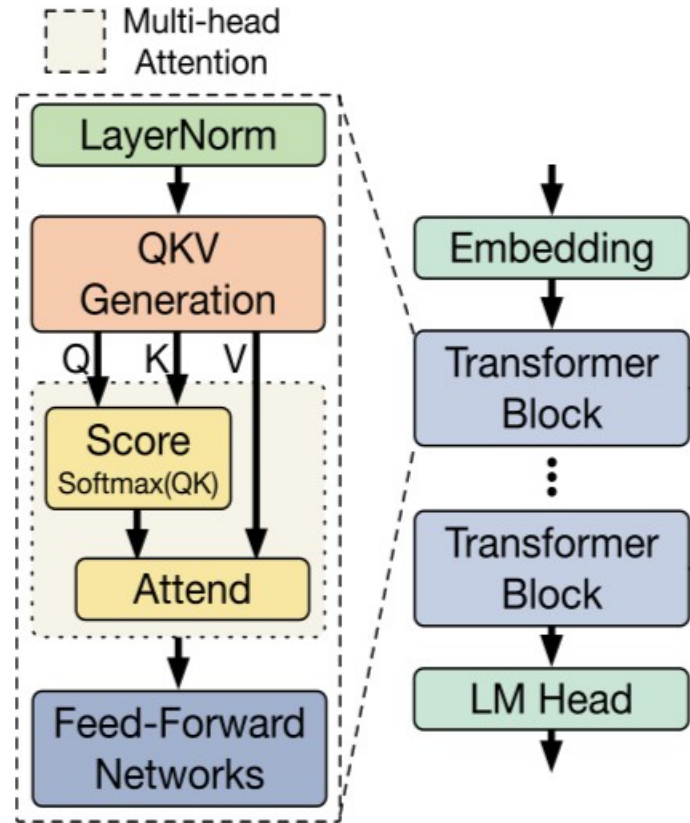


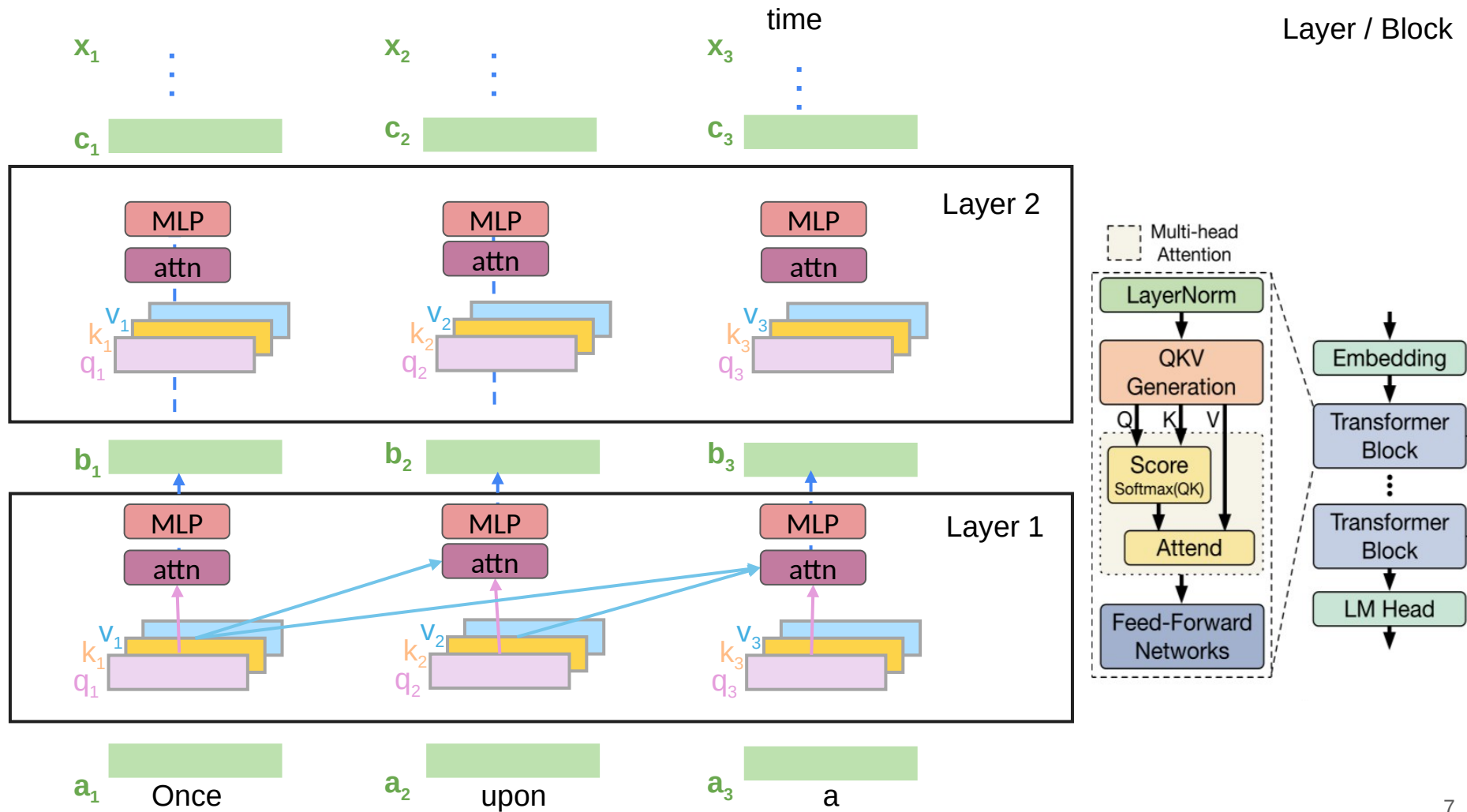
What is inference?

- Text generation
- Classification
- Translation



Transformers Architecture





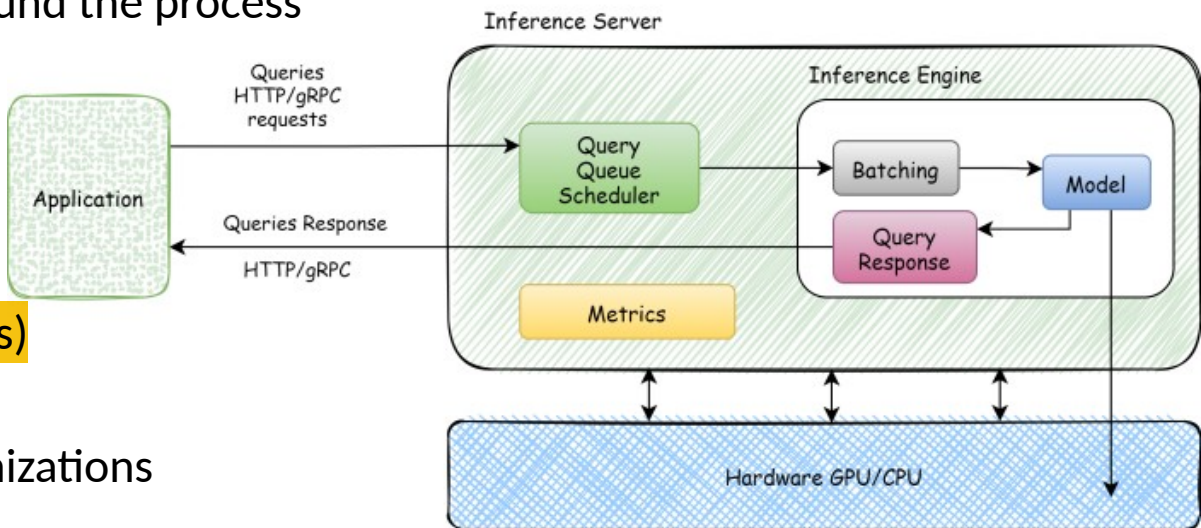
Background/Terminology

1. Inference engines

- Wrap systems concepts around the process of inference

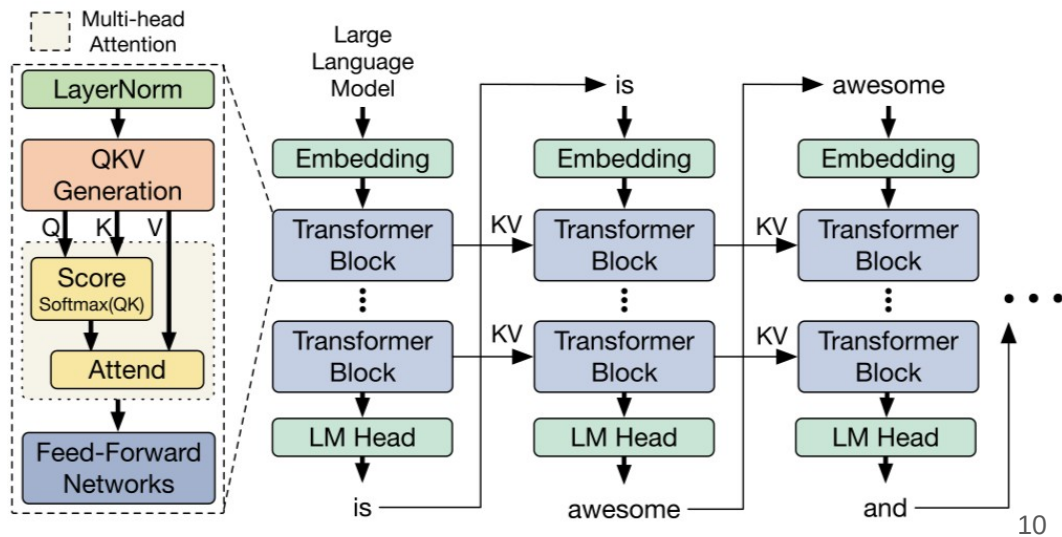
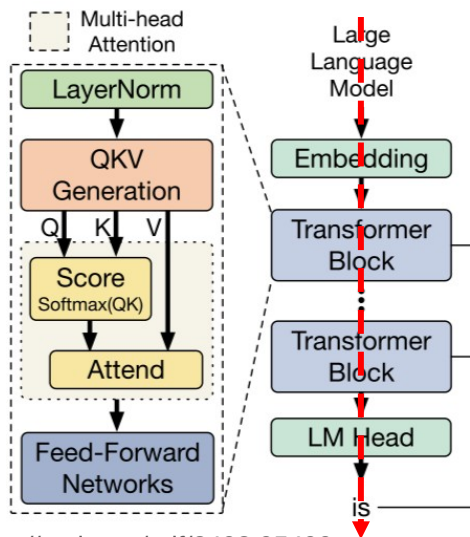
- Batching
- Streaming
- Fault management
- Parallelism (multi-GPUs)

- Optimizations:
 - Memory storage optimizations
 - Offloading
 - Speculative decoding
- Most popular open-source is vllm



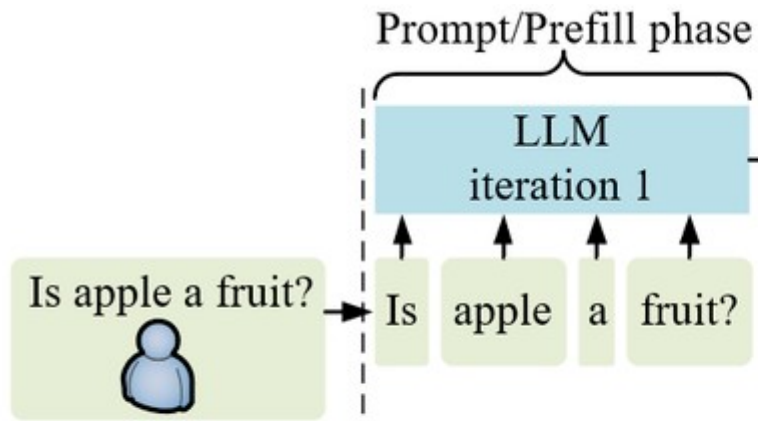
2. forward vs generate

- Forward: single forward pass through LLM
- Generate: Multiple tokens, until EOS or max tokens reached



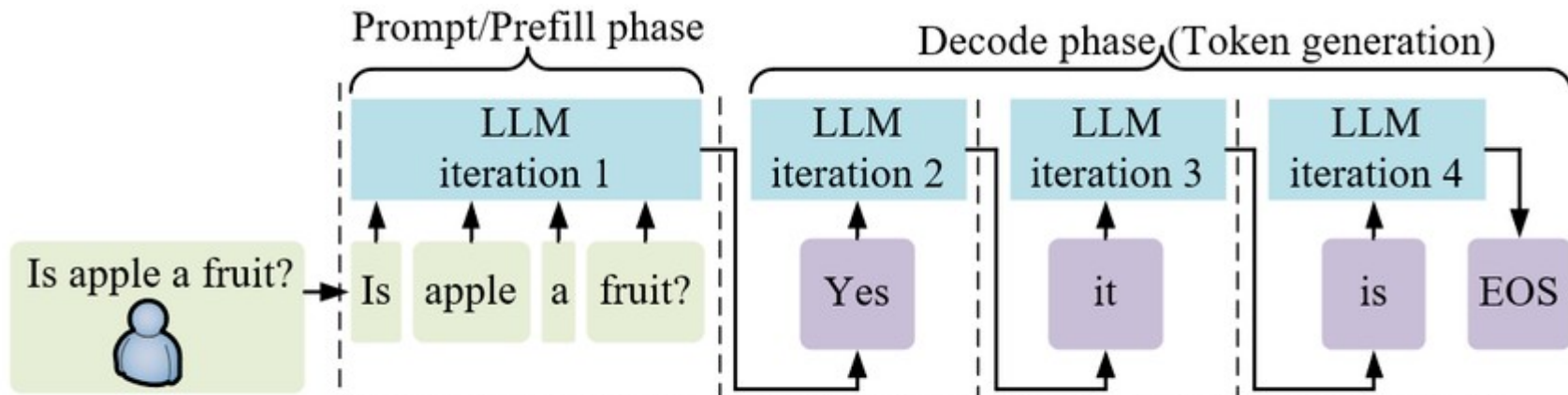
3. Prefill

- Prefill: input/prompt phase. Generation of the first output token (ingestion of the entire input in parallel)



3. Prefill vs decode

- Prefill: input/prompt phase. Generation of the first output token (ingestion of the entire input in parallel)
- Decode: output/generation phase. Decoding of subsequent output tokens (sequential)



3. Prefill vs decode

Parallel process

- Forward of all tokens happens in one iteration

Sequential Process

- Forward of one token in one iteration

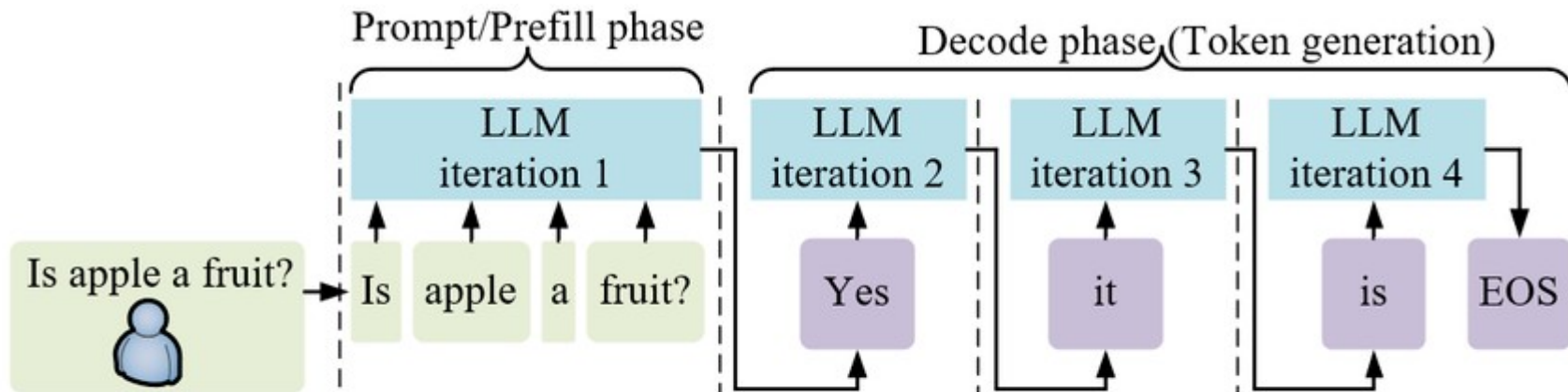
4. Metrics

- Throughput: Number of Tokens processed per second

- Latency:

- Time to First Token (prefill time)

- Time between Tokens (decode time)



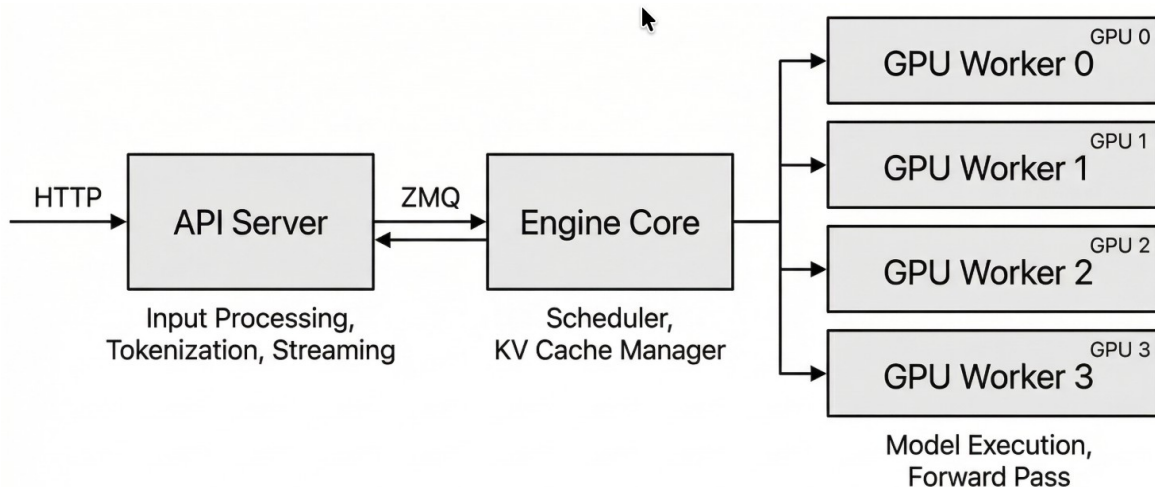
Core features

Core features of inference engines

Inference engines such as vllm etc. have many features/optimizations

We will look at 3 such techniques:

1. KV Caching
2. Prefix Caching



1. KV Caching

Why KV Caching?

1. What are K and V used for?

Self-attention calculation requires Q of current token and K&V of other tokens

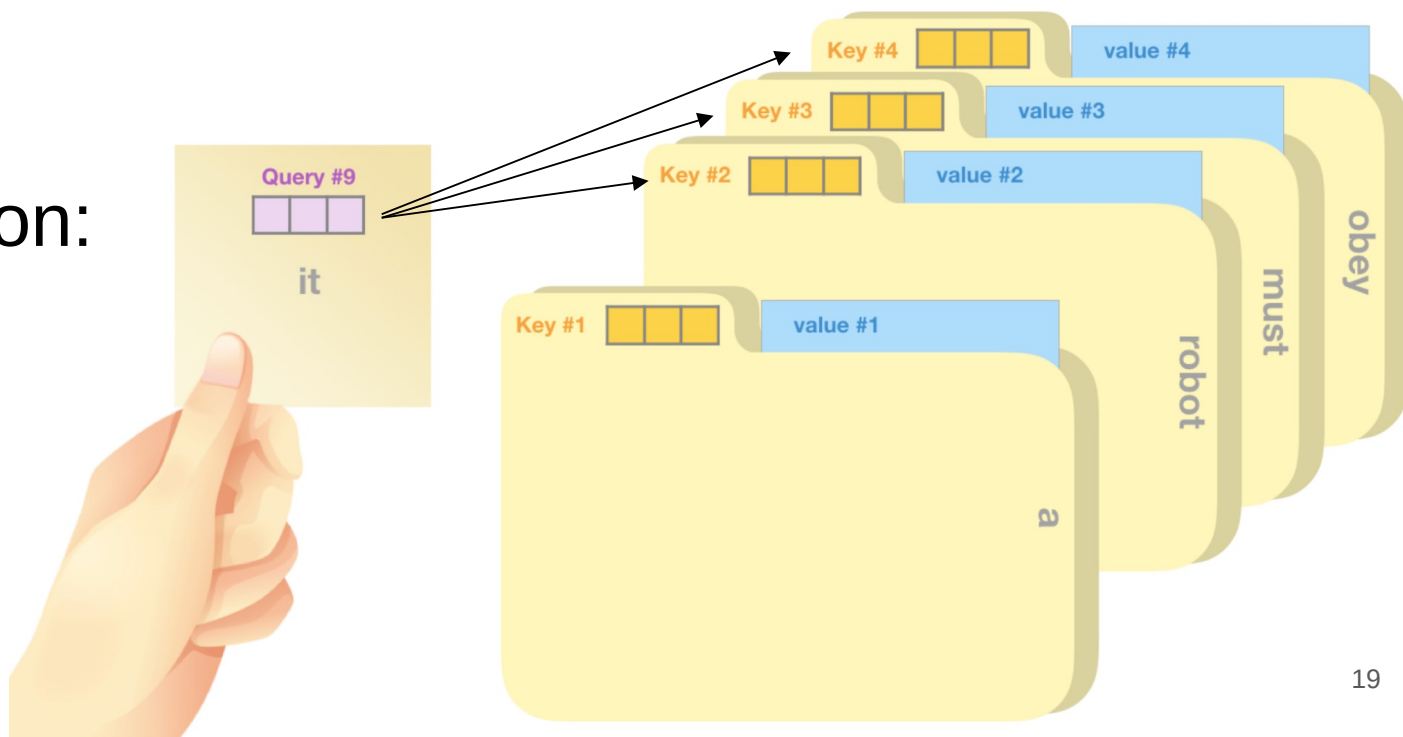
Causal self-attention requires K&V of all past tokens

Query: The query is a representation of the current word used to score against all the other words (using their keys). We only care about the query of the token we're currently processing.

Key: Key vectors are like labels for all the words in the segment. They're what we match against in our search for relevant words.

Value: Value vectors are actual word representations, once we've scored how relevant each word is, these are the values we add up to represent the current word.

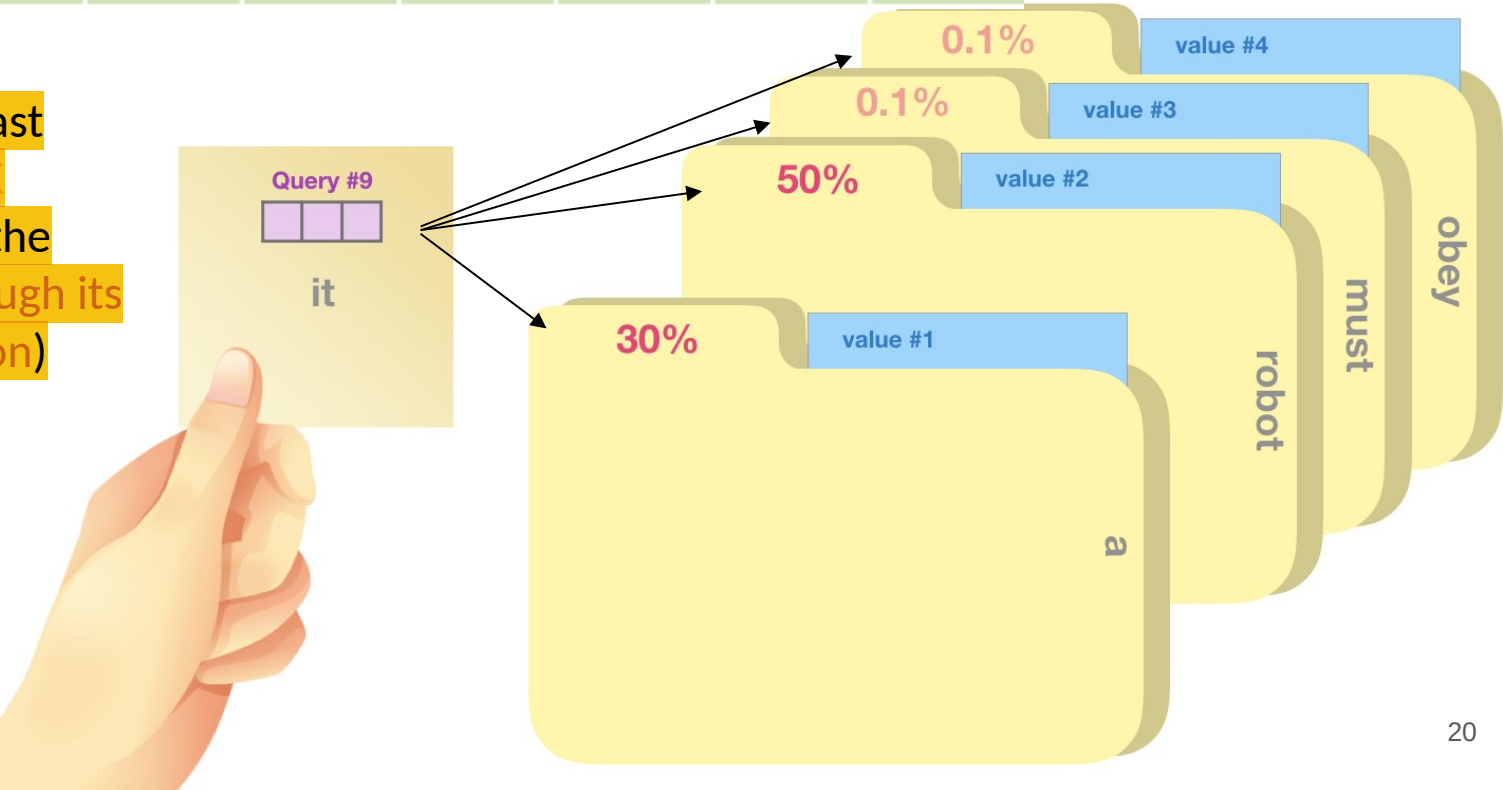
Goal of attention:
infusion of context





<s> a robot must obey the orders given it

Q.K is a score:
how relevant is a past
token (through its K
representation) to the
current token (through its
query representation)



Word	Value vector	Score	Value X Score
<S>		0.001	
a		0.3	
robot		0.5	
must		0.002	
Finally, each past token (through its Value Representation) is multiplied with the score and summed up to obtain a new representation of the current token		0.001	
		0.0003	
		0.005	
		0.002	
given		0.002	
it		0.19	
		Sum:	

Why KV Caching?

1. What are K and V used for?

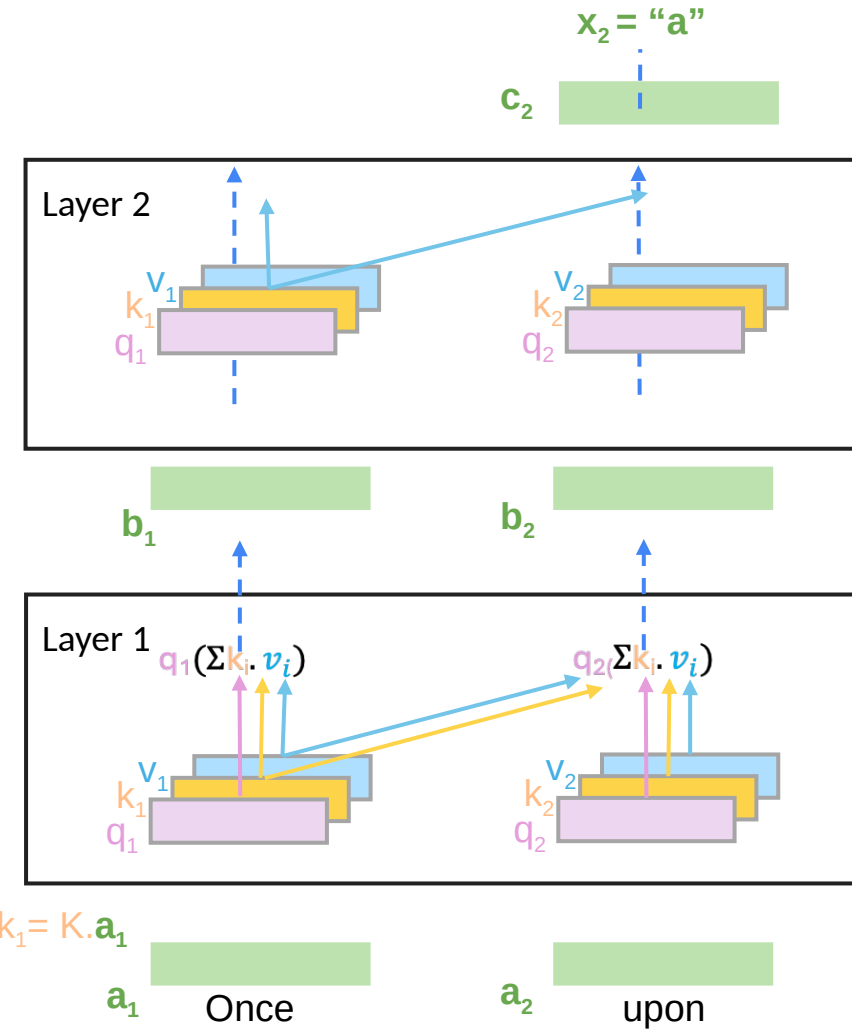
Self-attention calculation requires Q of current token and K&V of other tokens

Causal self-attention requires K&V of all past tokens

2. Why cache K and V?

Problem: It is recalculated again and again for every token

Idea: Generate and cache once, reuse for subsequent tokens



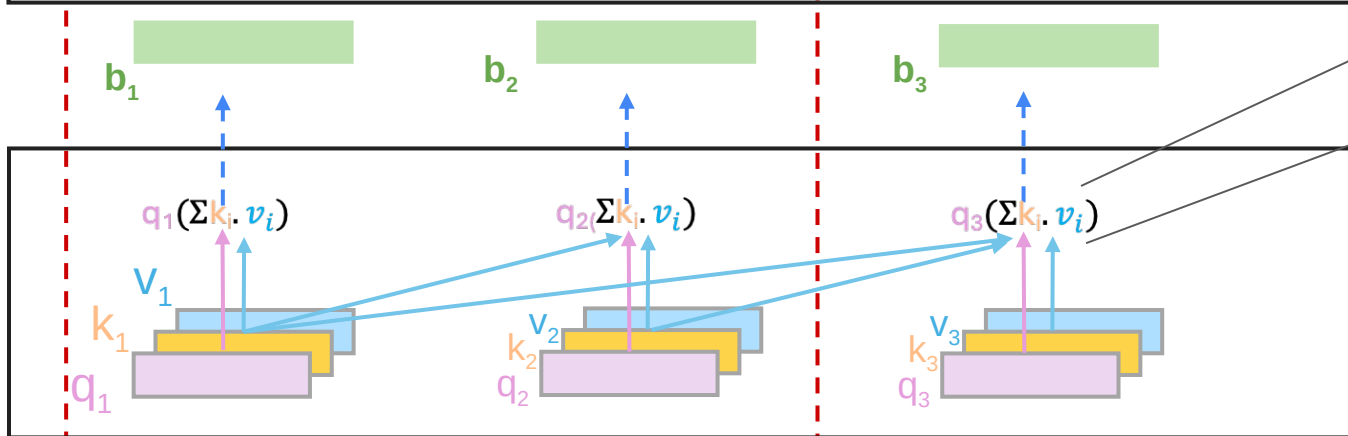
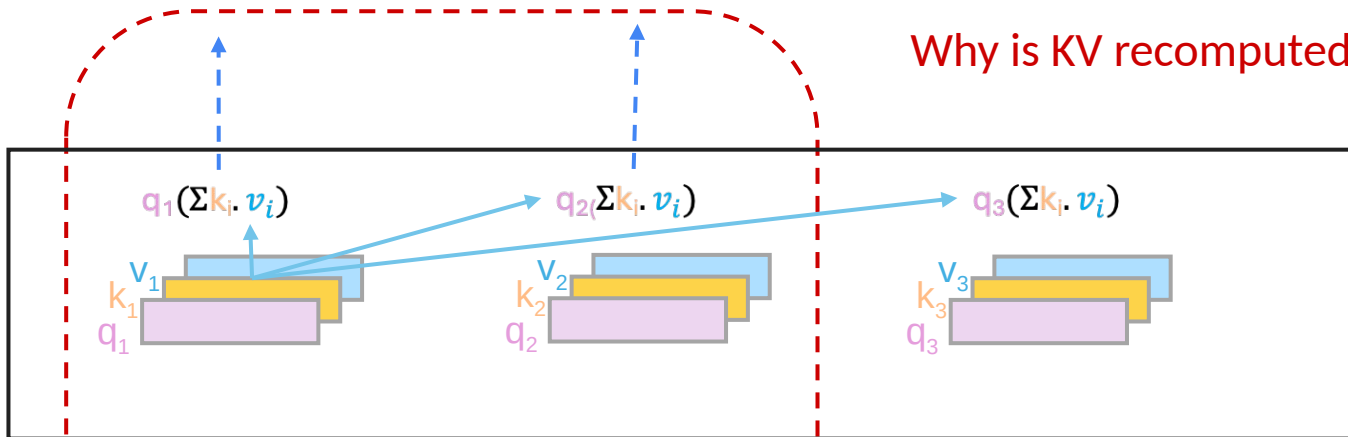
$$q_1 = Q \cdot a_1$$

b_1 is output after layer 1

For every token, only its q matters

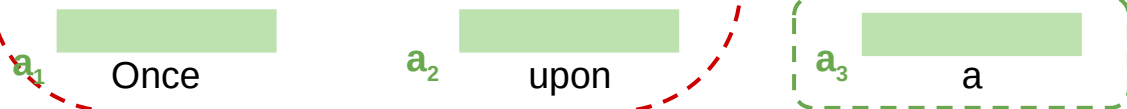
Causal attention: For every token, only its and past tokens' k, v matter

Why is KV recomputed?

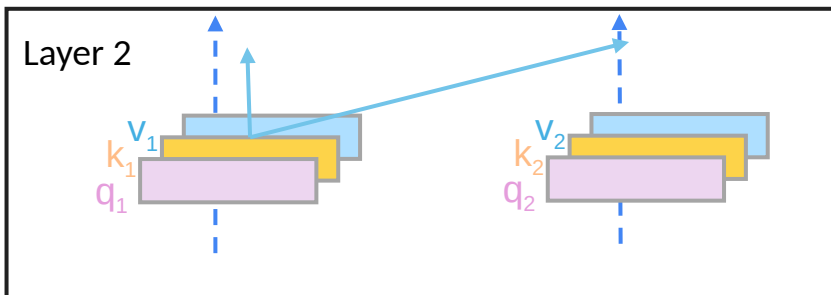


$q_3(\sum k_i \cdot v_i)$

1. We need the K and V of all tokens in the past, at every layer
2. This requires calculation of b_1, b_2 etc. for all past tokens

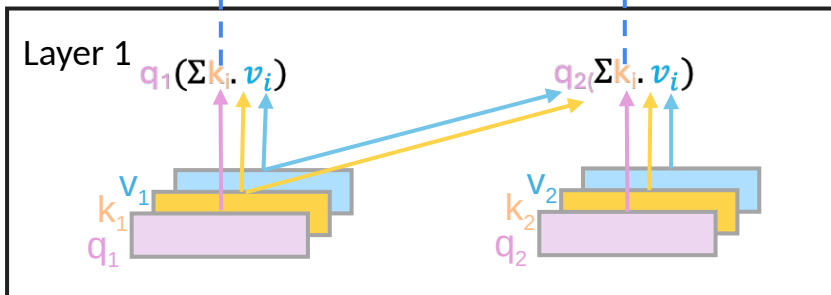


$x_2 = \text{"a"}$
 c_2



b_1

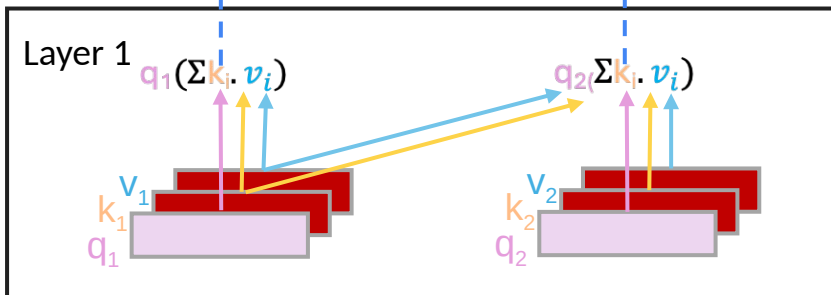
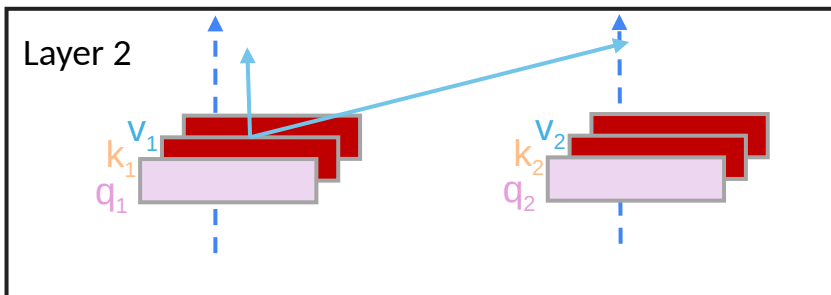
b_2



$k_1 = K \cdot a_1$
 a_1 Once

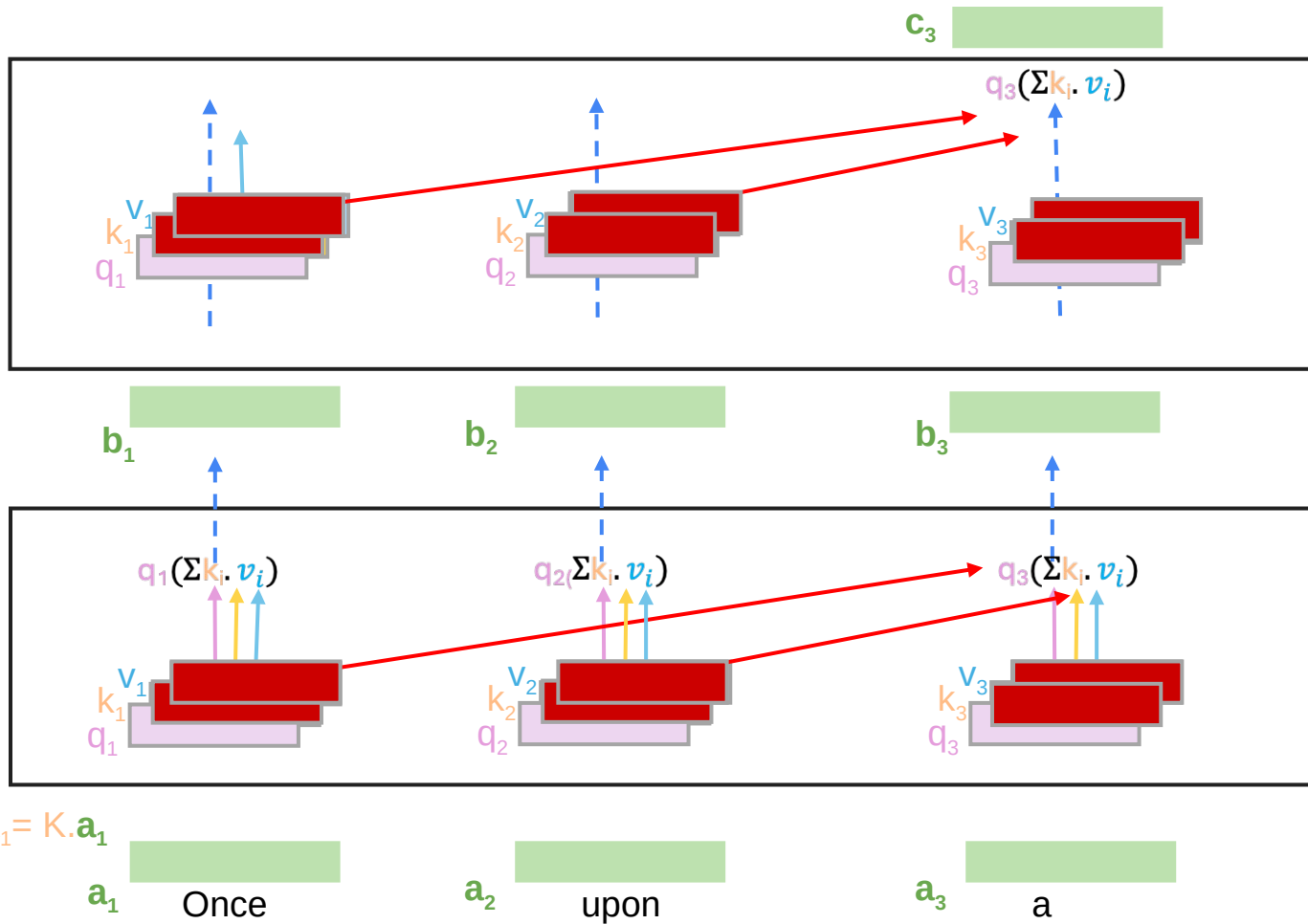
a_2 upon

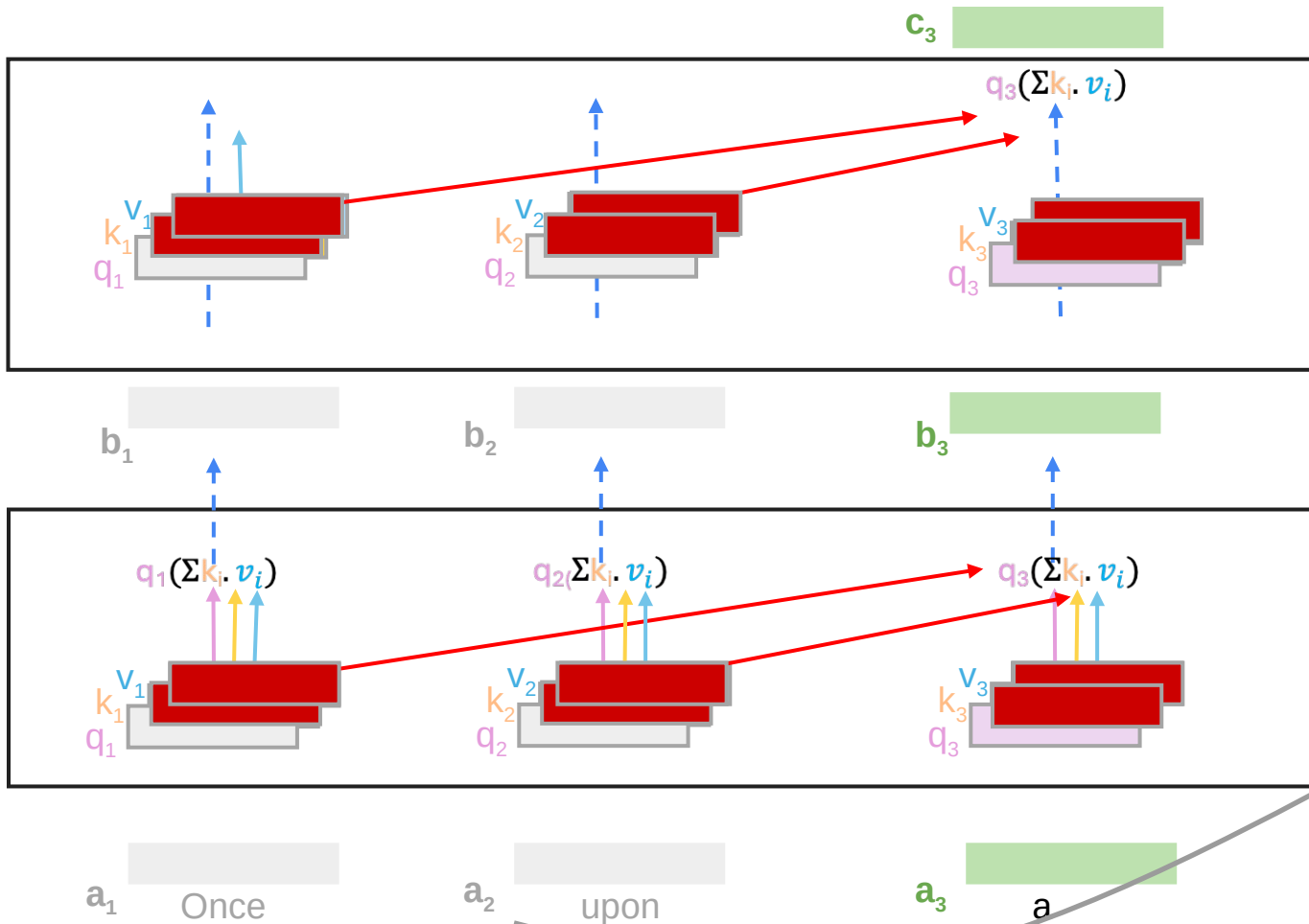
$x_2 = \text{"a"}$
 c_2



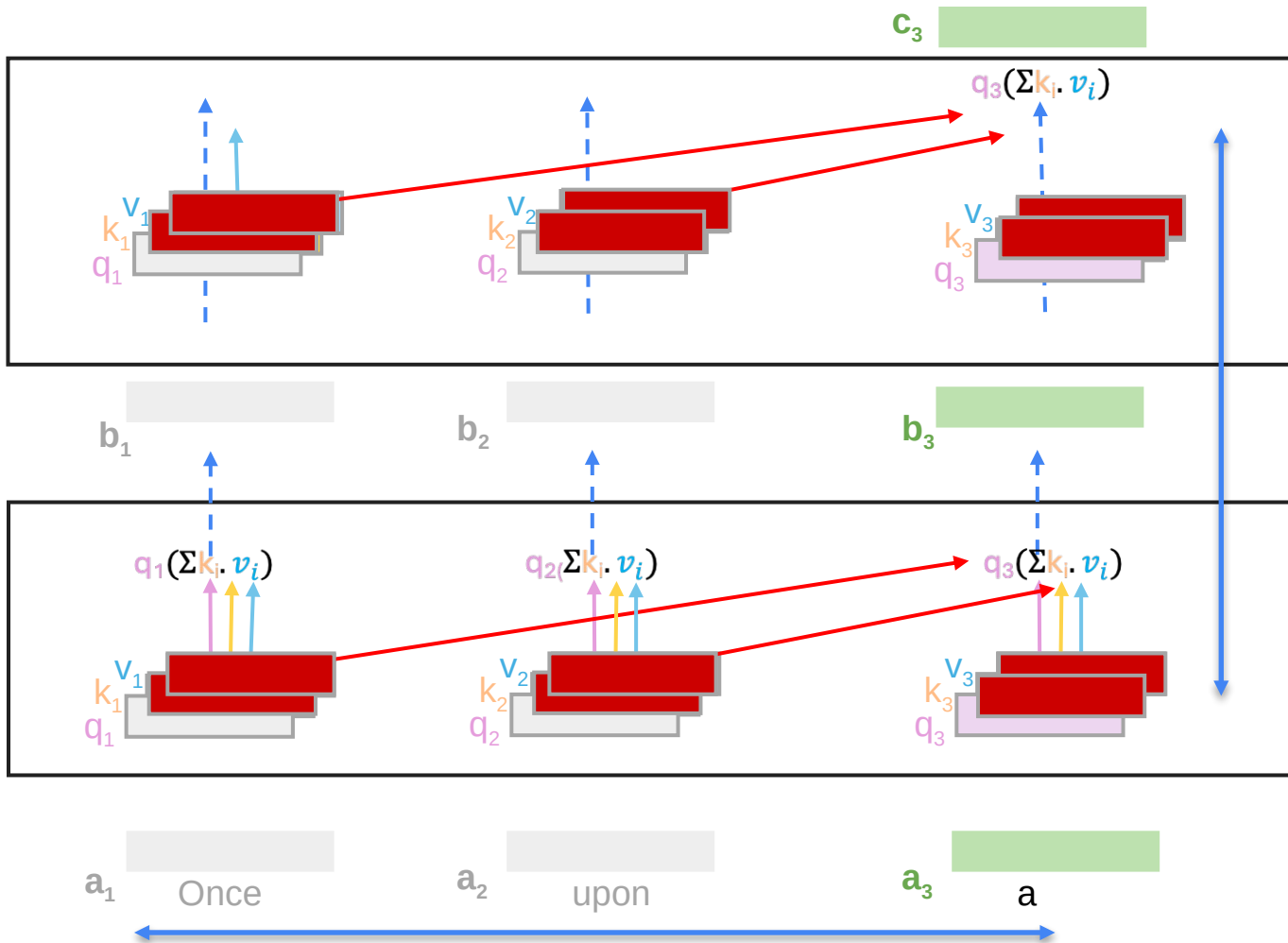
$k_1 = K \cdot a_1$
 a_1 Once

a_2 upon





We don't pass entire input for every decode run, only the last token



What is the size of the KV cache?

How many KVs are we storing?

Num layers x

Num tokens x

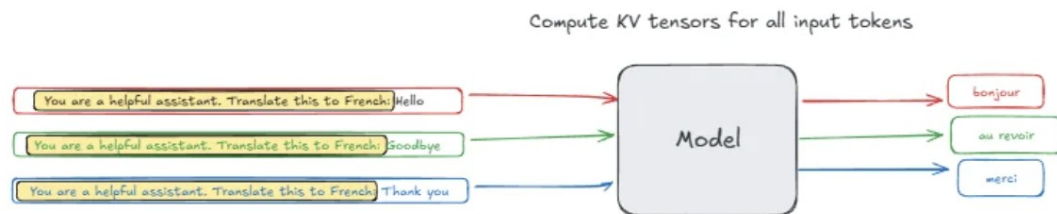
2

Size of K/V?:

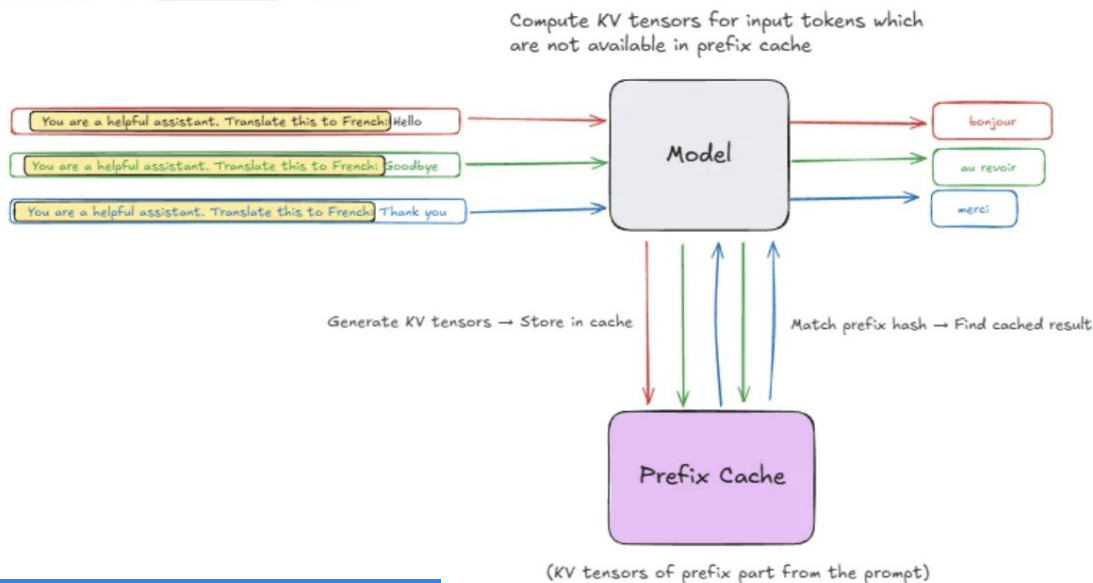
Embedding size

2. Prefix Caching

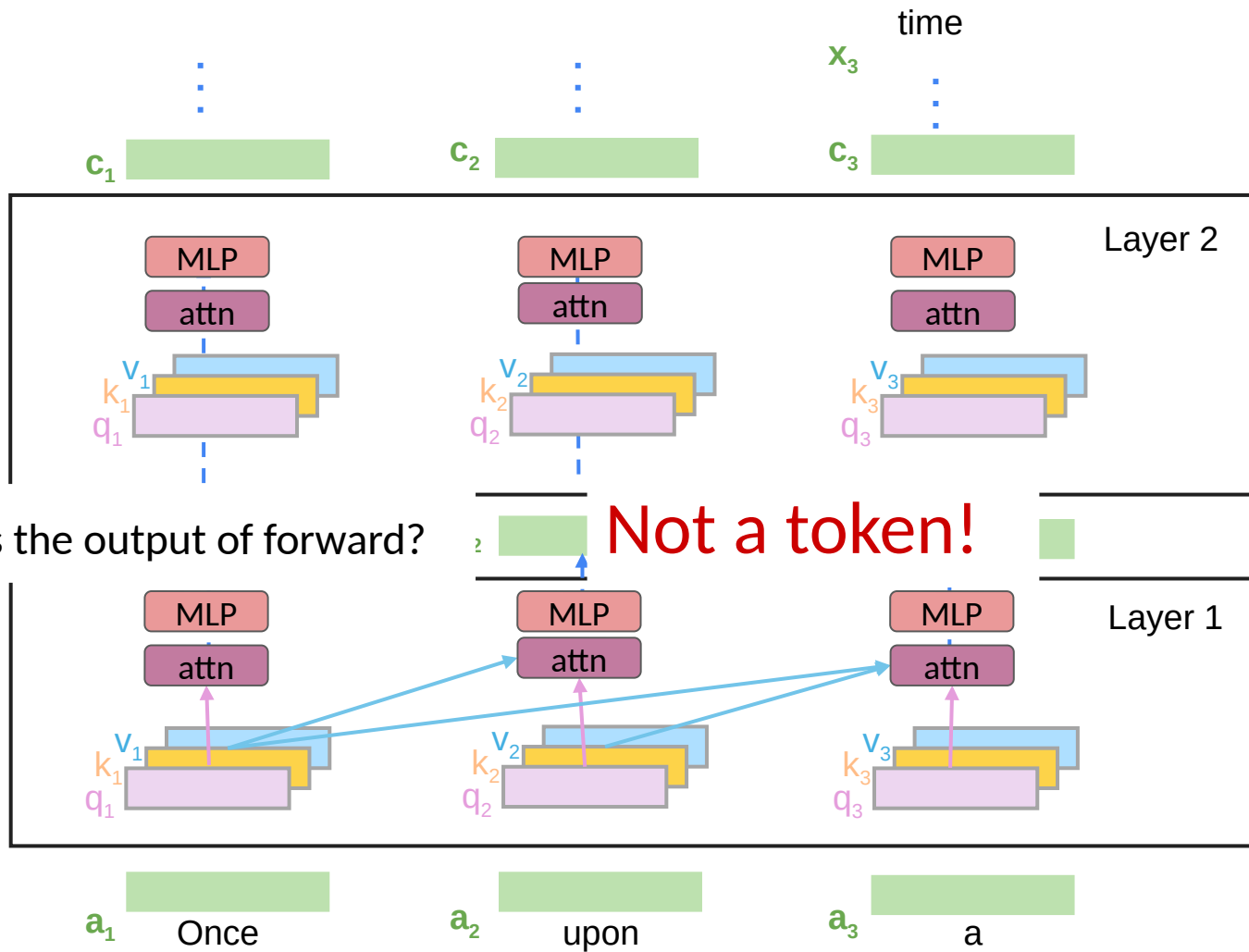
Without Prefix Cache :



With Prefix Cache :



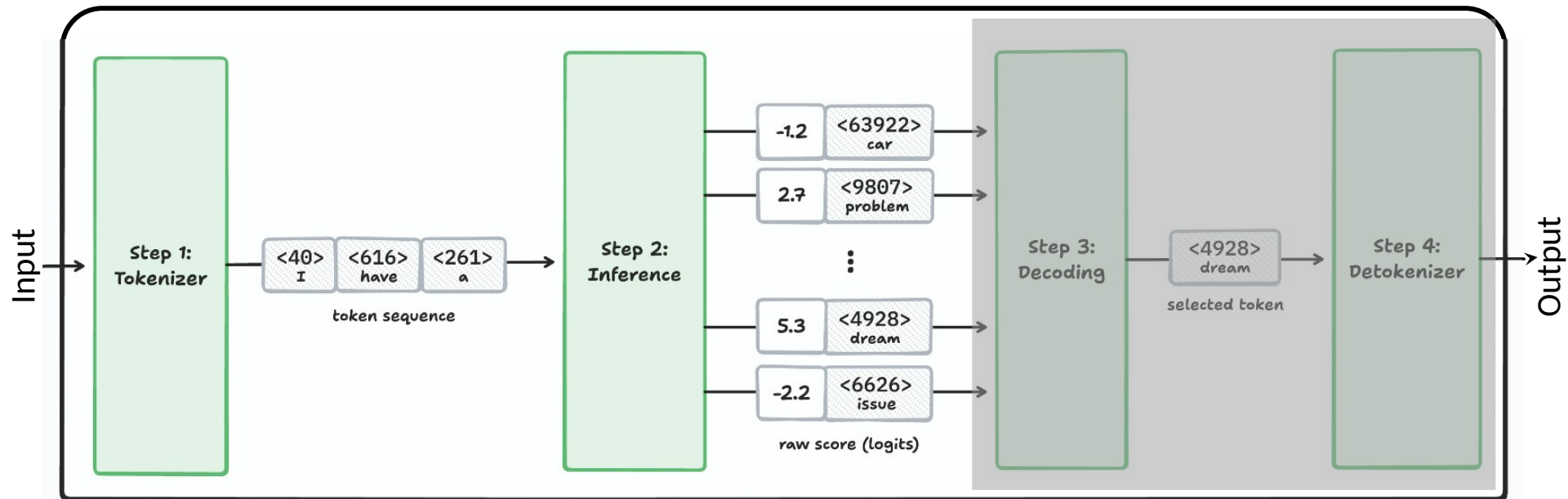
Decoding strategies



What is the output of forward?

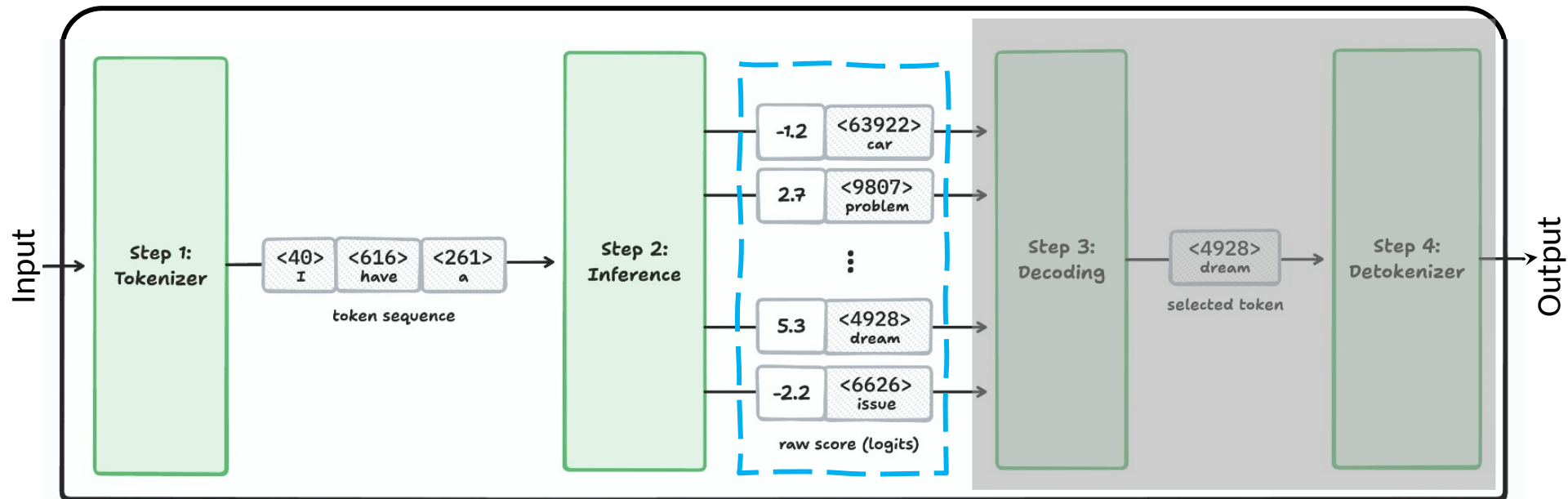
Not a token!

What is the output of forward?

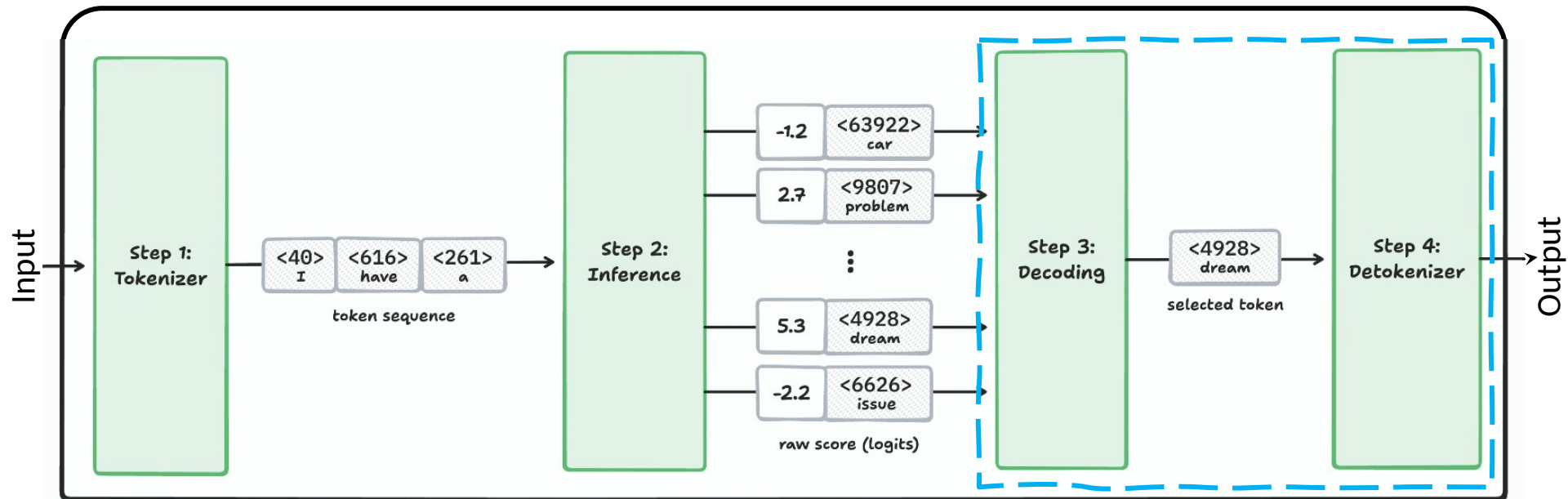


What is the output of forward?

Logits: A degree of similarity to each token in the vocabulary



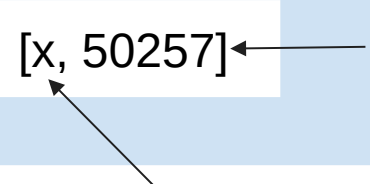
Decoding



Decoding Strategies

Which token to predict as response?

```
logits = model(inputs, positions)
print(logits.shape [x, 50257]
```



Vocab size

Batch size

How to convert these logits to a token?

Softmax and temperature

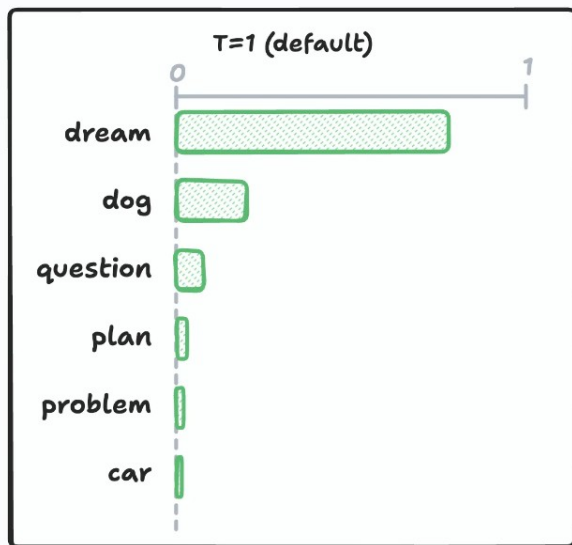
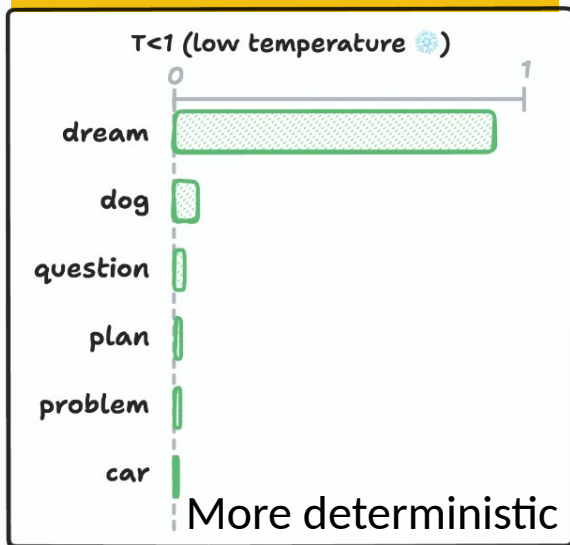
Logit (similarity score) -> Probability score (that adds up to 1)

This is done via a softmax normalization function which has a “temperature” parameter

which scales the logits value.

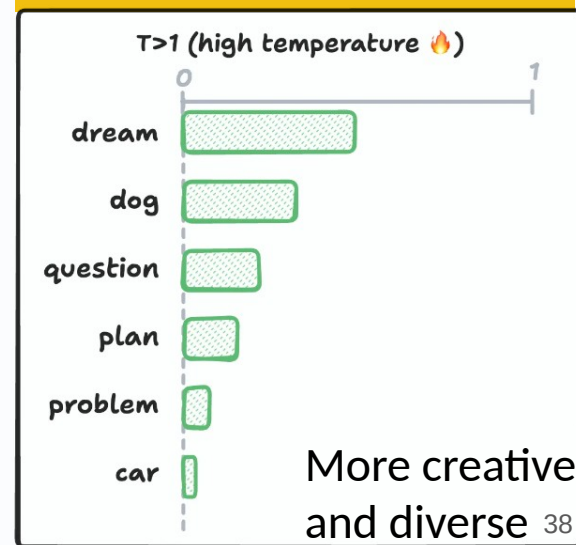
Low temperature: Gives more weight to already heavy ones

Makes values more extreme

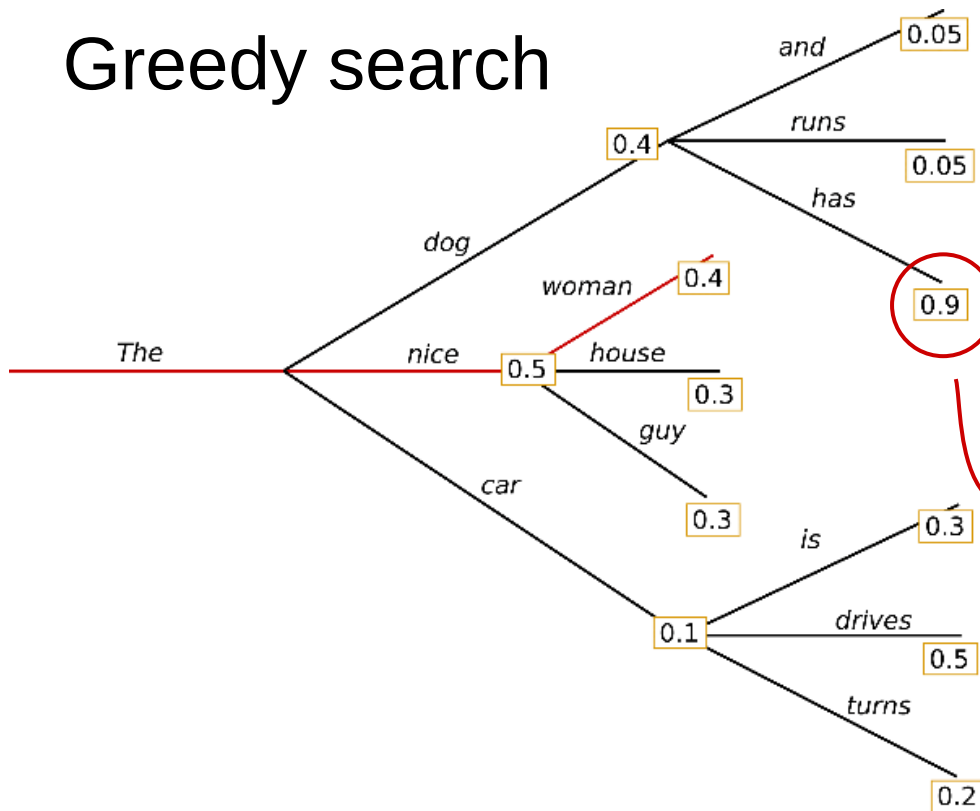


High temperature: Gives more weight to less heavy ones

Makes values more centered



Greedy search



Choose word with highest probability at every step

Problems:

1. It starts repeating itself after some time
2. Misses high prob. words hidden behind low prob. words

```
next_token = torch.argmax(logits, dim=-1, keepdim=True)
```

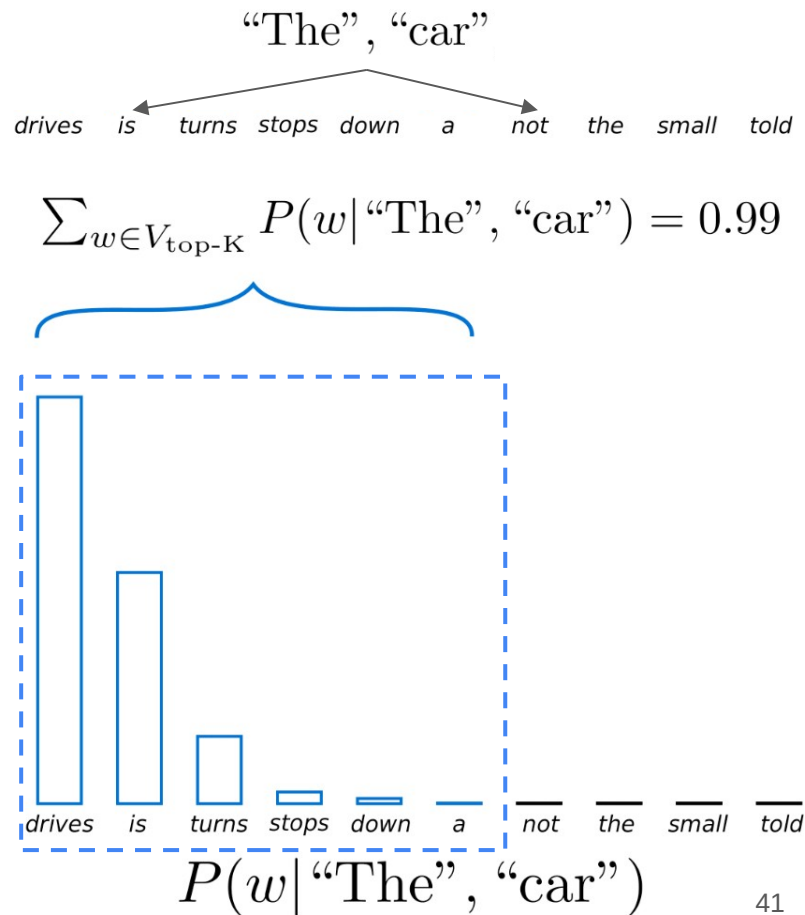
Greedy Search

> I enjoy walking with my cute dog

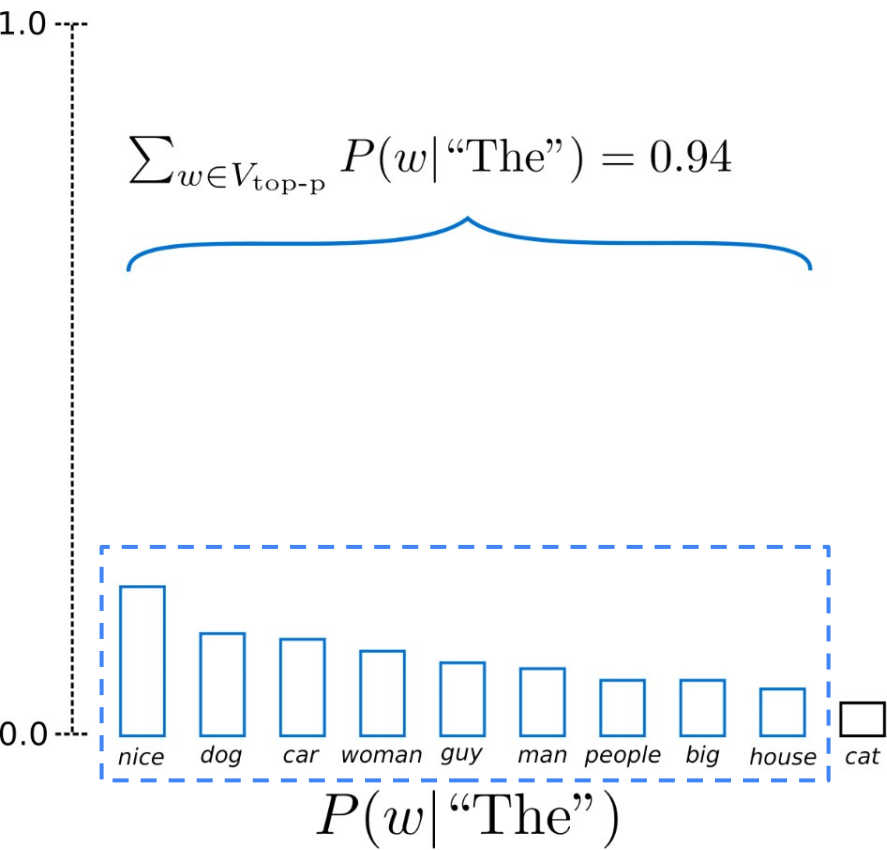
I enjoy walking with my cute dog, but I'm not sure if I'll ever be able to walk with my dog. I'm not sure if I'll ever be able to walk with my dog. I'm not sure

Top-K Sampling

- Randomly pick one of the output tokens
- But this could bias towards the long tail of irrelevant options
- Select only top-K of those and normalize the probability
- How to decide K?



Top-p sampling



First decide threshold probability p (0.94)

K is decided by the number of options that add up to p